



Scientific Programming Simplified with GNU Scientific Library

Since the advent of GSL, a lot of tools have been designed to make GSL more user friendly. In this article, I'll discuss some of the tools that can prove handy in scientific programming.

Before going on, I would like to do a quick recap of my previous two articles. The first was a basic introduction to GSL, its installation, and a newbie program in GSL. In my second article, I talked about writing portable programs, and errors in GSL. Now, I will assume that you have read my previous articles, and possibly tried the programs too. This article will cover the utilities provided for efficient programming. Although they are not compulsory, they can simplify your work to a great extent. I will first talk about tools that are installed along with GSL, and then later about tools that use GSL but need to be installed separately.

Installed tools

For the most commonly required utilities, like a random number generator, GSL has provided a few functions that will help us. Let's look at how to use some of these.

Random number generator

Many of us need to generate random numbers, for a variety of reasons. Some use different languages like R and Octave; others might write a program using GSL. To overcome this drawback, GSL gives you a

terminal command, *gsl-randist*, which generates 'n' random numbers from various distributions like a beta distribution, a gamma distribution, and so on. The usage (in a terminal) is: *gsl-randist seed n DIST param1 param2...*

```
$ gsl-randist 1246 5 gamma 1 2
0.851665
1.07302
1.52706
0.393288
9.64876
```

Even if you are not aware of the distribution parameters, just run the command without the parameters, and it will prompt you for the needed parameters:

```
$ gsl-randist seed n DIST
```

Making histograms

This is another important feature that is needed to know the frequency distributions of unknown data. The command *gsl-histogram* helps in counting the frequencies within the range of numbers given. Usage: *gsl-histogram xmin xmax [n]*. 'n' is the number of bins